

Code Review

Good code review requires human skills and technical skills alike. Therefore, alongside actual technical findings, I am also describing my thoughts and strategy. If this review is for LLM-generated code I would simply provide Raw Findings (see below).

Strategy

This PR has many issues. It is critical to approach the review without antagonizing the author and avoiding the impression of gating (many review rounds). It is a delicate balance to strike. My strategy would be to provide general direction in the first review pass and then provide details in the second pass.

Delivery

Ideally I would deliver both review passes in writing, with the exception of very junior staff or when there is additional information about the author (like higher sensitivity to feedback). In such a case I would first opt for a 30 minute call *before* publishing any feedback in writing.

Review Pass #1

The goal is to make the author realize as many faults in their code without overwhelming them. A natural place to start is to point to the test coverage, so that as many defects are fixed along the way as possible.

- **test_app:14** Great, thank you for adding a test! What do you think about extending the test suite to also cover more complex cases and corner cases (request/response failures, bad input, etc)?
- **app.py:17** I see you have changed the function that is called here. Is there any existing code that will become obsolete as a result?
- **npm_deps/package.py:8** Maybe worth considering caching results and parallelizing the queries? What do you think?
- **npm_deps/package.py:11** I see that old code uses `max_satisfying`, is the switch to `min_satisfying` deliberate or a typo?

Review Pass #2

With the second pass it would be important to address the remaining high severity issues. Critically we need to ensure correct error handling, safety and lack of dead code. If there are many changes that are still needed to reach that point I would schedule a short call to discuss the fixes and what I meant in the first review pass. If there are not so many issues remaining I would list them and ask if they could be fixed using questions and suggestive wording "What do you think of ...?", "Do you believe the code would improve if ...", "It seems to me that doing ... would help future maintainers", etc. some examples below:

- **tests/e2e/test_app.py:14** "Do you think that also covering the failure and retry mechanism makes sense here?"
- **npm_deps/package.py:8** "Would caching the results of get_package_version yield any noticeable performance gain?"
- ...

Post Review

Any remaining out of scope, pre-existing and low severity issues can be accepted as technical debt if the team agrees.

Raw Findings

The following list of findings are my raw notes of what I see could be improved in the PR. In that raw form they are not to be provided to another person.

npm_deps/package.py:8

- Missing exception handling. A single failed dependency request, or a wrong format being returned, causes complete request failure. Is that the desired behavior? (high)
- Default does not seem to be used. Why would we accept None? (low)
- Caching get_package result for specific name + range combination could improve performance. (high - at least caching)
- Missing docstring (medium)

npm_deps/package.py:9

- There already is a helper function in `package_request.py` that can be reused. (medium)
- `request_package` should also consider retry, exponential backoff etc. (high, pre-existing)
- Errors are not checked, with an immediate call to `.json()` (high)

`npm_deps/package.py:11`

- Should this be `max_satisfying`? (high)

`npm_deps/package.py:19`

- Should we parallelize here? Increases complexity but might improve performance. Depends on the overall context where the app runs. (low, pre-existing)
- Can we exclude cycles in dependencies? Otherwise we might end up with an infinite loop. (high)

`npm_deps/package.py:25`

- It seems `get_package_version` is not used. (high)

`tests/e2e/test_app.py:14 (high)`

- empty & invalid version format
- empty & invalid package name format
- actual transitive dependencies
- at least one other package name + version
- at least one non-existent package name
- at least one non-existent package version
- supplying version range
- simulating repeated npm request failure
- simulating invalid response format
- checking status codes returned depending on failure type
- retry mechanism for requests

`app.py:17`

- No validation of name and version inputs coming from the wire for early error detection. Fixed address is used internally so exploits are unlikely. (medium)
- Type hint removed without an obvious reason. (medium)

`app.py:18`

- Likely changes in this file become unnecessary if instead of adding code we alter the existing code in `npm_deps.package` (low - if dead code removed)